Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ
к лабораторной работе №2
на тему
«ЭЛЕМЕНТЫ КРИПТОГРАФИИ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ГЕРЧИК Артем Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
Вывод 6
Приложение А (Обязательное) Листинг программного кода 7

1 ПОСТАНОВКА ЗАДАЧИ

Целью данной лабораторной работы является получение практических навыков в области классической криптографии путём самостоятельной реализации симметричных шифров замены. Изучение подобных алгоритмов позволяет сформировать базовое понимание принципов защиты информации, оценить стойкость простых шифров и осознать их уязвимости. В качестве объектов реализации выбраны шифр Цезаря и шифр Виженера.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Шифр Цезаря основан на сдвиге каждого символа текста на фиксированное число позиций вдоль алфавита. Числовой ключ K задаёт величину этого сдвига: при шифровании каждая буква заменяется буквой, стоящей на K позиций правее, а при выходе за границу алфавита счёт продолжается с его начала. Дешифрование выполняется обратным сдвигом на то же значение. Поскольку один и тот же ключ применяется ко всему тексту, шифр относится к классу моноалфавитных подстановок и уязвим к частотному анализу. Пример шифрования и дешифрования текста с помощью шифра Цезаря представлен на рисунке 2.1.

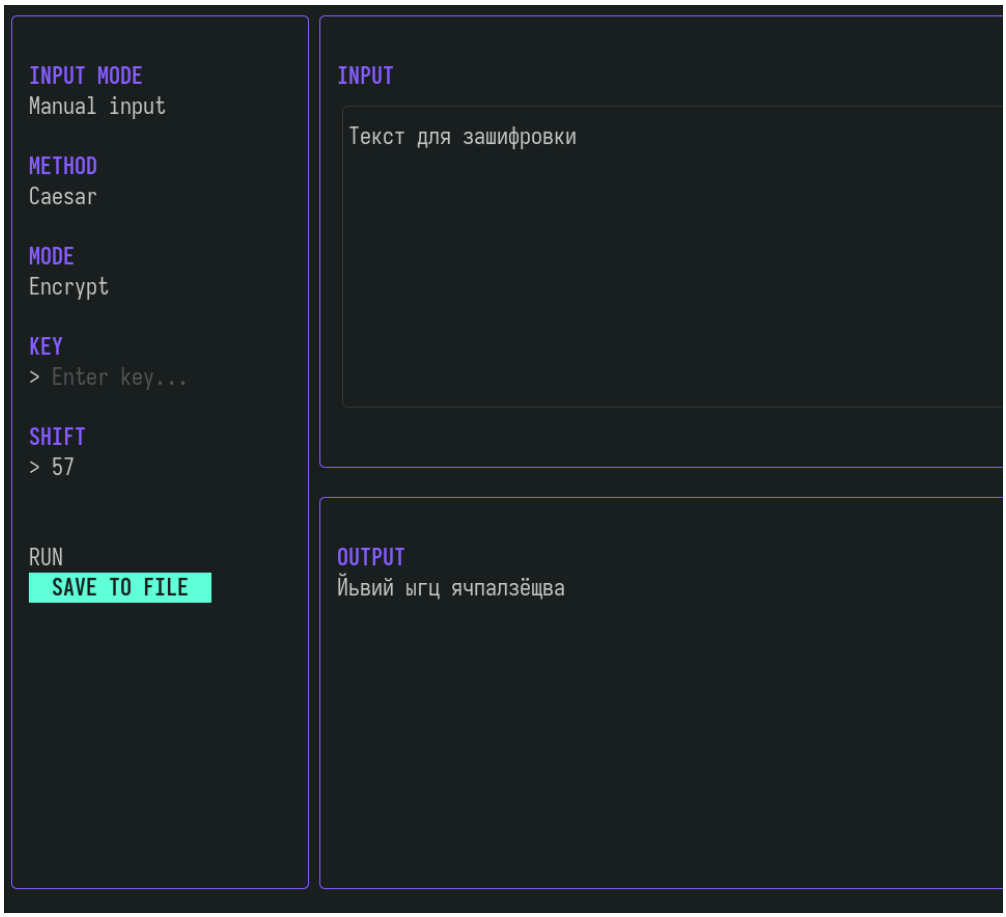

Рисунок 2.1 – Результат шифрования и дешифрования методом Цезаря, используя ручной ввод

Шифр Виженера является полиалфавитным обобщением шифра Цезаря. Вместо одного числового ключа используется ключевое слово, каждая буква которого задаёт индивидуальный сдвиг для соответствующей буквы текста. Ключевое слово циклически накладывается на текст: буквы ключевого слова поочерёдно применяются к буквам текста, и после последней буквы ключа счёт начинается заново. Счётчик позиции в ключе продвигается только при обработке литерных символов, благодаря чему знаки препинания и пробелы не

нарушают выравнивание ключа. В результате одна и та же буква в разных позициях шифруется по-разному, что существенно усложняет частотный анализ. Пример работы с шифром Виженера показан на рисунке 2.2.

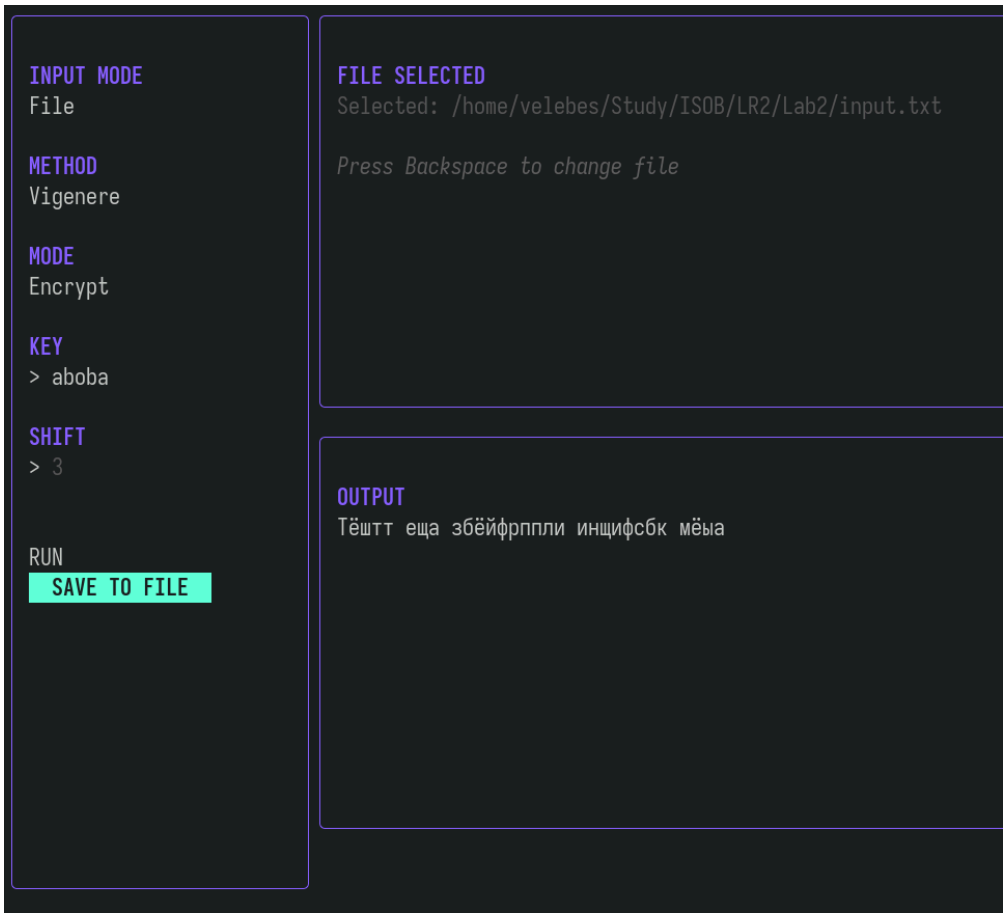

Рисунок 2.2 – Результат шифрования и дешифрования методом Виженера, используя ввод из файла

Программа реализована в виде интерактивного терминального приложения (*TUI*) на языке *Go*. Интерфейс разделён на функциональные зоны: панель настроек для выбора метода шифрования (Цезарь или Виженер) и режима работы (шифрование или дешифрование), а также области ввода и вывода данных. Навигация между элементами управления осуществляется с помощью клавиш *Tab* и *Shift+Tab*, что позволяет оперативно изменять параметры алгоритма. Поддерживается как ручной ввод текста в многострочное поле, так и выбор входного файла через встроенный файловый менеджер. Результат обработки отображается в реальном времени, при этом предусмотрена возможность сохранения итогового текста в отдельный файл через всплывающее диалоговое окно. Логика шифрования обеспечивает корректную обработку русского и латинского алфавитов с сохранением регистра, знаков препинания и пробелов.

ВЫВОД

В ходе выполнения лабораторной работы реализованы шифры Цезаря и Виженера на языке *Go* с поддержкой русского и латинского алфавитов, сохранением регистра и нелитерных символов. Разработан интерактивный терминальный интерфейс (*TUI*) с поддержкой ручного ввода текста, выбора файла через встроенный файловый менеджер и сохранения результата. Тестирование подтвердило корректность обоих алгоритмов: последовательное шифрование и дешифрование полностью восстанавливает исходный текст.

Вместе с тем анализ реализованных шифров выявил их принципиальные уязвимости. Шифр Цезаря имеет всего столько ключей, сколько букв в алфавите, что делает перебор тривиальным. Шифр Виженера устойчивее при длинном ключе, однако при известной или угаданной длине ключа сводится к набору независимых шифров Цезаря. Таким образом, изученные алгоритмы представляют историческую и учебную ценность, но не пригодны для защиты реальных данных и служат лишь отправной точкой для понимания современной криптографии.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Код метода *shiftRune*

```
package crypto

const (
  enLower = "abcdefghijklmnopqrstuvwxyz"
  enUpper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
  ruLower = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"
  ruUpper = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
)

func shiftRune(r rune, k int) rune {
  findShift := func(alphabet string) rune {
    runes := []rune(alphabet)
    n := len(runes)
    for i, a := range runes {
      if r == a {
        newID := (i + k) % n
        if newID < 0 {
          newID += n
        }
        return runes[newID]
      }
    }
    return r
  }

  switch {
  case (r >= 'a' && r <= 'z'):
    return findShift(enLower)
  case (r >= 'A' && r <= 'Z'):
    return findShift(enUpper)
  case (r >= 'a' && r <= 'я') || r == 'ё':
    return findShift(ruLower)
  case (r >= 'A' && r <= 'Я') || r == 'Ё':
    return findShift(ruUpper)
  default:
    return r
  }
}
```

Листинг А.2 – Код алгоритма Цезаря

```
package crypto

func Caesar(str []rune, key int) []rune {
  res := make([]rune, len(str))
  for i, c := range str {
    res[i] = shiftRune(c, key)
  }
```

```
  return res
}
```

Листинг А.3 – Код алгоритма Виженера

```
package crypto

func Vigenere(str []rune, key []rune, decrypt bool) []rune {
  res := make([]rune, len(str))
  keyID := 0
  for i, r := range str {
    var shift int
    var found bool

    k := key[keyID%len(key)]

    for i, a := range []rune(enLower) {
      if a == k || []rune(enUpper)[i] == k {
        shift = i
        found = true
        break
      }
    }
    if !found {
      for i, a := range []rune(ruLower) {
        if a == k || []rune(ruUpper)[i] == k {
          shift = i
          found = true
          break
        }
      }
    }

    if decrypt {
      shift = -shift
    }

    newR := shiftRune(r, shift)
    res[i] = newR

    keyID++
  }
  return res
}
```

Листинг А.4 – Код терминального интерфейса

```
package main

import (
  "go-cipher/crypto"
  "os"
  "strconv"
  "time"
```

```
  "github.com/charmbracelet/bubbles/key"
  tea "github.com/charmbracelet/bubbletea"
  "github.com/charmbracelet/lipgloss"
)

func (m model) Init() tea.Cmd {
  return m.Filepicker.Init()
}

func (m model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
  var cmds []tea.Cmd

  switch msg := msg.(type) {
  case tea.WindowSizeMsg:
    m.Width = msg.Width
    m.Height = msg.Height

    layout := NewLayout(msg.Width, msg.Height)

    m.Input.SetWidth(layout.TextareaInputWidth)
    m.Input.SetHeight(layout.TextareaInputHeight)
    m.Output.SetWidth(layout.TextareaOutputWidth)
    m.Output.SetHeight(layout.TextareaOutputHeight)
    m.Filepicker.SetHeight(layout.FilepickerHeight)
    m.SaveFilename.Width = 34

  case statusMsg:
    m.Status = ""
    return m, nil

  case tea.KeyMsg:
    // Save Popup Logic
    if m.ShowSavePopup {
      switch msg.String() {
      case "esc":
        m.ShowSavePopup = false
        m.SaveFilename.Blur()
        return m, nil
      case "enter":
        filename := m.SaveFilename.Value()
        if filename == "" {
          filename = "output.txt"
        }

        err := os.WriteFile(filename, []byte(m.Output.Value()), 0o644)
        if err != nil {
          m.Status = "Error: " + err.Error()
        } else {
          m.Status = "Saved to: " + filename
        }

        m.ShowSavePopup = false
        m.SaveFilename.Blur()

        return m, tea.Tick(3*time.Second, func(t time.Time) tea.Msg {
          return statusMsg("")
        })
```

```
  default:
    var cmd tea.Cmd
    m.SaveFilename, cmd = m.SaveFilename.Update(msg)
    return m, cmd
  }
}

// Global Bindings
switch {
case key.Matches(msg, m.keymap.quit):
  m.Input.Blur()
  m.Output.Blur()
  m.Key.Blur()
  m.Shift.Blur()
  return m, tea.Quit
case key.Matches(msg, m.keymap.next):
  m.Focus = (m.Focus + 1) % FocusCount
  m.skipInactive(1)
  m.updateFocus()
  return m, m.Filepicker.Init()
case key.Matches(msg, m.keymap.prev):
  m.Focus = (m.Focus - 1 + FocusCount) % FocusCount
  m.skipInactive(-1)
  m.updateFocus()
  return m, m.Filepicker.Init()
}

// Filepicker specific
if m.InputMode == File && m.Focus == FocusFilepicker {
  if !m.FileConfirmed {
    var cmd tea.Cmd
    m.Filepicker, cmd = m.Filepicker.Update(msg)
   if didSelect, path := m.Filepicker.DidSelectFile(msg); didSelect {
      m.SelectedFile = path
      m.FileConfirmed = true
    }
    return m, cmd
  }
  if msg.String() == "backspace" {
    m.FileConfirmed = false
    m.SelectedFile = ""
    return m, m.Filepicker.Init()
  }
  return m, nil
}

// Other focusable elements
switch {
case key.Matches(msg, m.keymap.copy):
  m.Input.SetValue(m.Output.Value())
  m.Output.SetValue("")
  m.InputMode = ManualInput
  m.Focus = FocusInput
  m.updateFocus()
  return m, nil
case key.Matches(msg, m.keymap.clearInput):
  m.Input.SetValue("")
```

```
    m.Focus = FocusInput
    m.updateFocus()
    return m, nil
  case m.Focus == FocusInputMode && msg.String() == "enter":
    m.InputMode = (m.InputMode + 1) % 2
    m.FileConfirmed = false
    m.SelectedFile = ""
    m.updateFocus()
    return m, m.Filepicker.Init()
  case m.Focus == FocusMethod && msg.String() == "enter":
    m.Method = (m.Method + 1) % 2
    m.skipInactive(1)
    m.updateFocus()
    return m, nil
  case m.Focus == FocusMode && msg.String() == "enter":
    m.Mode = (m.Mode + 1) % 2
    return m, nil
  case m.Focus == FocusBtn && msg.String() == "enter":
    m.Output.SetValue("")
    var res []rune
    decrypt := (m.Mode == Decrypt)

    var inputSource []rune
    if m.InputMode == File {
      if m.SelectedFile == "" {
        m.Output.SetValue("No file selected")
        return m, nil
      }
      content, err := os.ReadFile(m.SelectedFile)
      if err != nil {
        m.Output.SetValue("Error reading file: " + err.Error())
        return m, nil
      }
      inputSource = []rune(string(content))
    } else {
      inputSource = []rune(m.Input.Value())
    }

    switch m.Method {
    case Caesar:
      key, err := strconv.Atoi(m.Shift.Value())
      if err != nil {
        m.Output.SetValue("Shift should be positive number")
        return m, nil
      }
      if decrypt {
        key = -key
      }
      res = crypto.Caesar(inputSource, key)
    case Vigenere:
      if _, err := strconv.Atoi(m.Key.Value()); err == nil {
        m.Output.SetValue("Key cannot be a number")
        return m, nil
      }
    res = crypto.Vigenere(inputSource, []rune(m.Key.Value()), decrypt)
    }
```

```
        m.Output.SetValue(string(res))
        return m, nil
      case m.Focus == FocusSaveBtn && msg.String() == "enter":
        m.ShowSavePopup = true
        m.SaveFilename.SetValue("")
        m.SaveFilename.Focus()
        return m, nil
      }
    }

    var cmd tea.Cmd

    if m.Focus == FocusInput {
      m.Input, cmd = m.Input.Update(msg)
      cmds = append(cmds, cmd)
    }
    if m.Focus == FocusFilepicker {
      m.Filepicker, cmd = m.Filepicker.Update(msg)
      cmds = append(cmds, cmd)
    }
    if m.Focus == FocusKey {
      m.Key, cmd = m.Key.Update(msg)
      cmds = append(cmds, cmd)
    }
    if m.Focus == FocusShift {
      m.Shift, cmd = m.Shift.Update(msg)
      cmds = append(cmds, cmd)
    }

    return m, tea.Batch(cmds...)
  }

  func (m model) View() string {
    layout := NewLayout(m.Width, m.Height)

    inputModeLabel := labelStyle.Render("INPUT MODE")
    inputModeValue := "File"
    if m.InputMode == ManualInput {
      inputModeValue = "Manual input"
    }
    if m.Focus == FocusInputMode {
      inputModeValue = selectedStyle.Render(inputModeValue) + " (Enter)"
    }

    methodLabel := labelStyle.Render("METHOD")
    methodValue := "Caesar"
    if m.Method == Vigenere {
      methodValue = "Vigenere"
    }
    if m.Focus == FocusMethod {
      methodValue = selectedStyle.Render(methodValue) + " (Enter)"
    }

    modeLabel := labelStyle.Render("MODE")
    modeValue := "Encrypt"
    if m.Mode == Decrypt {
      modeValue = "Decrypt"
```

```
  }
  if m.Focus == FocusMode {
    modeValue = selectedStyle.Render(modeValue) + " (Enter)"
  }

  keyLabel := labelStyle.Render("KEY")
  shiftLabel := labelStyle.Render("SHIFT")

  runBtnLabel := "RUN"
  if m.Focus == FocusBtn {
    runBtnLabel = buttonStyle.Render(runBtnLabel)
  }

  label := "SAVE TO FILE"
  if m.Focus == FocusSaveBtn {
    label = saveBtnStyle.Render(label)
  }
  saveBtnLabel := label

  leftPanel := lipgloss.JoinVertical(
    lipgloss.Left,
    inputModeLabel,
    inputModeValue,
    "",
    methodLabel,
    methodValue,
    "",
    modeLabel,
    modeValue,
    "",
    keyLabel,
    m.Key.View(),
    "",
    shiftLabel,
    m.Shift.View(),
    "",
    "",
    runBtnLabel,
    saveBtnLabel,
  )

 leftPanel = panelStyle(LeftPanelWidth, layout.LeftPanelHeight).Render(lipglos
lipgloss.Top, leftPanel))

  inputPanelStyle := lipgloss.NewStyle().
    Height(layout.TopPanelHeight).
    Width(layout.RightPanelWidth).
    Border(lipgloss.RoundedBorder()).
    BorderForeground(borderColor).
    Padding(1)

  outputPanelStyle := lipgloss.NewStyle().
    Height(layout.BottomPanelHeight).
    Width(layout.RightPanelWidth).
    Border(lipgloss.RoundedBorder()).
    BorderForeground(borderColor).
    Padding(1)
```

```
  inputTitle := "INPUT"
  inputContent := m.Input.View()

  if m.InputMode == File {
    if m.FileConfirmed {
      inputTitle = "FILE SELECTED"

      style := selectedStyle
      if m.Focus != FocusFilepicker {
        style = lipgloss.NewStyle().Foreground(lipgloss.Color("240"))
      }

      selectedLine := style.Render("Selected: " + m.SelectedFile)
      hintLine := lipgloss.NewStyle().
        Foreground(lipgloss.Color("241")).
        Italic(true).
        Render("Press Backspace to change file")
      inputContent = lipgloss.JoinVertical(lipgloss.Left, selectedLine,
"", hintLine)
    } else {
      inputTitle = "FILE PICKER"
      inputContent = m.Filepicker.View()
    }
  }

                                          inputPanel                    :=
inputPanelStyle.Render(lipgloss.PlaceVertical(layout.TopPanelHeight,
lipgloss.Top, labelStyle.Render(inputTitle)+"\n"+inputContent))

  outputView := m.Output.View()
  if m.Status != "" {
    outputView = lipgloss.NewStyle().
      Foreground(lipgloss.Color("86")).
      Bold(true).
      Render(m.Status)
  }

 outputPanel := outputPanelStyle.Render(lipgloss.PlaceVertical(layout.BottomPa
lipgloss.Top, labelStyle.Render("OUTPUT")+"\n"+outputView))

  rightPanel := lipgloss.JoinVertical(0, inputPanel, outputPanel)

  baseView := lipgloss.JoinHorizontal(0, leftPanel, rightPanel)

  if m.ShowSavePopup {
    popupContent := lipgloss.JoinVertical(
      lipgloss.Left,
      labelStyle.Render("SAVE OUTPUT TO FILE"),
      "",
      "Filename:",
      m.SaveFilename.View(),
      "",
      lipgloss.NewStyle().
        Foreground(lipgloss.Color("241")).
        Italic(true).
        Render("Enter to save  •  Esc to cancel"),
```

```
    )
    popup := popupStyle.Render(popupContent)
    return lipgloss.Place(
      m.Width, m.Height,
      lipgloss.Center, lipgloss.Center,
      popup,
      lipgloss.WithWhitespaceChars(" "),
      lipgloss.WithWhitespaceForeground(lipgloss.Color("237")),
    )
  }

  return baseView
}

func (m *model) skipInactive(dir int) {
  for {
    inactive := false
    if m.InputMode == ManualInput && m.Focus == FocusFilepicker {
      inactive = true
    }
    if m.InputMode == File && m.Focus == FocusInput {
      inactive = true
    }
    if m.Method == Caesar && m.Focus == FocusKey {
      inactive = true
    }
    if m.Method == Vigenere && m.Focus == FocusShift {
      inactive = true
    }

    if !inactive {
      break
    }
    m.Focus = (m.Focus + dir + FocusCount) % FocusCount
  }
}

func (m *model) updateFocus() {
  m.Input.Blur()
  m.Key.Blur()
  m.Shift.Blur()
  m.SaveFilename.Blur()

  switch m.Focus {
  case FocusInput:
    m.Input.Focus()
  case FocusKey:
    m.Key.Focus()
  case FocusShift:
    m.Shift.Focus()
  }

  if m.InputMode == File {
    m.Input.Blur()
  }

  switch m.Method {
```

```
  case Caesar:
    m.Key.SetValue("")
    m.Key.Blur()
  case Vigenere:
    m.Shift.SetValue("")
    m.Shift.Blur()
  }
}
```